Recall that lossless join is an essential condition for a decomposition, to avoid loss of information. We are therefore forced to give up either BCNF or dependency preservation. In Section 7.7 we present an alternative normal form, called **third normal form**, which is a small relaxation of BCNF; the motivation for using third normal form is that there is always a dependency preserving decomposition into third normal form.

There are situations where there is more than one way to decompose a schema into BCNF. Some of these decompositions may be dependency preserving, while others may not. For instance, suppose we have a relation schema $R(A, B, C)$ with the functional dependencies $A \rightarrow B$ and $B \rightarrow C$. From this set we can derive the further dependency $A \rightarrow C$. If we used the dependency $A \rightarrow B$ (or equivalently, $A \rightarrow C$) to decompose R, we would end up with two relations $R1(A, B)$ and $R2(A, C)$; the dependency $B \rightarrow C$ would not be preserved.

If instead we used the dependency $B \rightarrow C$ to decompose R, we would end up with two relations $R1(A, B)$ and $R2(B, C)$, which are in BCNF, and the decomposition is also dependency preserving. Clearly the decomposition into $R1(A, B)$ and $R2(B, C)$ is preferable. In general, the database designer should therefore look at alternative decompositions, and pick a dependency preserving decomposition where possible.

7.7 Third Normal Form

As we saw earlier, there are relational schemas where a BCNF decomposition cannot be dependency preserving. For such schemas, we have two alternatives if we wish to check if an update violates any functional dependencies:

- Pay the extra cost of computing joins to test for violations.
- Use an alternative decomposition, third normal form (3NF), which we present below, which makes testing of updates cheaper. Unlike BCNF, 3NF decompositions may contain some redundancy in the decomposed schema.

We shall see that it is always possible to find a lossless-join, dependency-preserving decomposition that is in 3NF. Which of the two alternatives to choose is a design decision to be made by the database designer on the basis of the application requirements.

7.7.1 Definition

BCNF requires that all nontrivial dependencies be of the form $\alpha \rightarrow \beta$, where α is a superkey. 3NF relaxes this constraint slightly by allowing nontrivial functional dependencies whose left side is not a superkey.

A relation schema R is in **third normal form (3NF)** with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency.
- α is a superkey for R.

- Each attribute A in $\beta - \alpha$ is contained in a candidate key for R.

Note that the third condition above does not say that a single candidate key should contain all the attributes in $\beta - \alpha$; each attribute A in $\beta - \alpha$ may be contained in a *different* candidate key.

The first two alternatives are the same as the two alternatives in the definition of BCNF. The third alternative of the 3NF definition seems rather unintuitive, and it is not obvious why it is useful. It represents, in some sense, a minimal relaxation of the BCNF conditions that helps ensure that every schema has a dependency-preserving decomposition into 3NF. Its purpose will become more clear later, when we study decomposition into 3NF.

Observe that any schema that satisfies BCNF also satisfies 3NF, since each of its functional dependencies would satisfy one of the first two alternatives. BCNF is therefore a more restrictive constraint than is 3NF.

The definition of 3NF allows certain functional dependencies that are not allowed in BCNF. A dependency $\alpha \rightarrow \beta$ that satisfies only the third alternative of the 3NF definition is not allowed in BCNF, but is allowed in 3NF.[1]

Let us return to our *Banker-schema* example (Section 7.6). We have shown that this relation schema does not have a dependency-preserving, lossless-join decomposition into BCNF. This schema, however, turns out to be in 3NF. To see that it is, we note that {*customer-name, branch-name*} is a candidate key for *Banker-schema*, so the only attribute not contained in a candidate key for *Banker-schema* is *banker-name*. The only nontrivial functional dependencies of the form

$$\alpha \rightarrow \textit{banker-name}$$

include {*customer-name, branch-name*} as part of α. Since {*customer-name, branch-name*} is a candidate key, these dependencies do not violate the definition of 3NF.

As an optimization when testing for 3NF, we can consider only functional dependencies in the given set F, rather than in F^+. Also, we can decompose the dependencies in F so that their right-hand side consists of only single attributes, and use the resultant set in place of F.

Given a dependency $\alpha \rightarrow \beta$, we can use the same attribute-closure–based technique that we used for BCNF to check if α is a superkey. If α is not a superkey, we have to verify whether each attribute in β is contained in a candidate key of R; this test is rather more expensive, since it involves finding candidate keys. In fact, testing for 3NF has been shown to be NP-hard; thus, it is very unlikely that there is a polynomial time complexity algorithm for the task.

7.7.2 Decomposition Algorithm

Figure 7.14 shows an algorithm for finding a dependency-preserving, lossless-join decomposition into 3NF. The set of dependencies F_c used in the algorithm is a canoni-

1. These dependencies are examples of **transitive dependencies** (see Exercise 7.25). The original definition of 3NF was in terms of transitive dependencies. The definition we use is equivalent but easier to understand.

```
let F_c be a canonical cover for F;
i := 0;
for each functional dependency α → β in F_c do
   if none of the schemas R_j, j = 1, 2, ..., i contains α β
      then begin
               i := i + 1;
               R_i := α β;
           end
if none of the schemas R_j, j = 1, 2, ..., i contains a candidate key for R
   then begin
           i := i + 1;
           R_i := any candidate key for R;
      end
return (R_1, R_2, ..., R_i)
```

Figure 7.14 Dependency-preserving, lossless-join decomposition into 3NF.

cal cover for *F*. Note that the algorithm considers the set of schemas $R_j, j = 1, 2, \ldots, i$; initially $i = 0$, and in this case the set is empty.

To illustrate the algorithm of Figure 7.14, consider the following extension to the *Banker-schema* in Section 7.6:

Banker-info-schema = (*branch-name*, *customer-name*, *banker-name*, *office-number*)

The main difference here is that we include the banker's office number as part of the information. The functional dependencies for this relation schema are

banker-name → *branch-name office-number*
customer-name branch-name → *banker-name*

The **for** loop in the algorithm causes us to include the following schemas in our decomposition:

Banker-office-schema = (*banker-name*, *branch-name*, *office-number*)
Banker-schema = (*customer-name*, *branch-name*, *banker-name*)

Since *Banker-schema* contains a candidate key for *Banker-info-schema*, we are finished with the decomposition process.

The algorithm ensures the preservation of dependencies by explicitly building a schema for each dependency in a canonical cover. It ensures that the decomposition is a lossless-join decomposition by guaranteeing that at least one schema contains a candidate key for the schema being decomposed. Exercise 7.19 provides some insight into the proof that this suffices to guarantee a lossless join.

This algorithm is also called the **3NF synthesis algorithm**, since it takes a set of dependencies and adds one schema at a time, instead of decomposing the initial schema repeatedly. The result is not uniquely defined, since a set of functional dependencies

can have more than one canonical cover, and, further, in some cases the result of the algorithm depends on the order in which it considers the dependencies in F_c.

If a relation R_i is in the decomposition generated by the synthesis algorithm, then R_i is in 3NF. Recall that when we test for 3NF, it suffices to consider functional dependencies whose right-hand side is a single attribute. Therefore, to see that R_i is in 3NF, you must convince yourself that any functional dependency $\gamma \rightarrow B$ that holds on R_i satisfies the definition of 3NF. Assume that the dependency that generated R_i in the synthesis algorithm is $\alpha \rightarrow \beta$. Now, B must be in α or β, since B is in R_i and $\alpha \rightarrow \beta$ generated R_i. Let us consider the three possible cases:

- B is in both α and β. In this case, the dependency $\alpha \rightarrow \beta$ would not have been in F_c since B would be extraneous in β. Thus, this case cannot hold.
- B is in β but not α. Consider two cases:
  - γ is a superkey. The second condition of 3NF is satisfied.
  - γ is not a superkey. Then α must contain some attribute not in γ. Now, since $\gamma \rightarrow B$ is in F^+, it must be derivable from F_c by using the attribute closure algorithm on γ. The derivation could not have used $\alpha \rightarrow \beta$— if it had been used, α must be contained in the attribute closure of γ, which is not possible, since we assumed γ is not a superkey. Now, using $\alpha \rightarrow (\beta - \{B\})$ and $\gamma \rightarrow B$, we can derive $\alpha \rightarrow B$ (since $\gamma \subseteq \alpha\beta$, and γ cannot contain B because $\gamma \rightarrow B$ is nontrivial). This would imply that B is extraneous in the right-hand side of $\alpha \rightarrow \beta$, which is not possible since $\alpha \rightarrow \beta$ is in the canonical cover F_c. Thus, if B is in β, then γ must be a superkey, and the second condition of 3NF must be satisfied.
- B is in α but not β.
  Since α is a candidate key, the third alternative in the definition of 3NF is satisfied.

Interestingly, the algorithm we described for decomposition into 3NF can be implemented in polynomial time, even though testing a given relation to see if it satisfies 3NF is NP-hard.

7.7.3 Comparison of BCNF and 3NF

Of the two normal forms for relational-database schemas, 3NF and BCNF, there are advantages to 3NF in that we know that it is always possible to obtain a 3NF design without sacrificing a lossless join or dependency preservation. Nevertheless, there are disadvantages to 3NF: If we do not eliminate all transitive relations schema dependencies, we may have to use null values to represent some of the possible meaningful relationships among data items, and there is the problem of repetition of information.

As an illustration of the null value problem, consider again the *Banker-schema* and its associated functional dependencies. Since *banker-name* $\rightarrow$ *branch-name*, we may want to represent relationships between values for *banker-name* and values for *branch-name* in our database. If we are to do so, however, either there must be a corresponding value for *customer-name*, or we must use a null value for the attribute *customer-name*.

customer-name	*banker-name*	*branch-name*
Jones	Johnson	Perryridge
Smith	Johnson	Perryridge
Hayes	Johnson	Perryridge
Jackson	Johnson	Perryridge
Curry	Johnson	Perryridge
Turner	Johnson	Perryridge

Figure 7.15 An instance of *Banker-schema*.

As an illustration of the repetition of information problem, consider the instance of *Banker-schema* in Figure 7.15. Notice that the information indicating that Johnson is working at the Perryridge branch is repeated.

Recall that our goals of database design with functional dependencies are:

1. BCNF
2. Lossless join
3. Dependency preservation

Since it is not always possible to satisfy all three, we may be forced to choose between BCNF and dependency preservation with 3NF.

It is worth noting that SQL does not provide a way of specifying functional dependencies, except for the special case of declaring superkeys by using the **primary key** or **unique** constraints. It is possible, although a little complicated, to write assertions that enforce a functional dependency (see Exercise 7.15); unfortunately, testing the assertions would be very expensive in most database systems. Thus even if we had a dependency-preserving decomposition, if we use standard SQL we would not be able to efficiently test a functional dependency whose left-hand side is not a key.

Although testing functional dependencies may involve a join if the decomposition is not dependency preserving, we can reduce the cost by using materialized views, which many database systems support. Given a BCNF decomposition that is not dependency preserving, we consider each dependency in a minimum cover F_c that is not preserved in the decomposition. For each such dependency $\alpha \rightarrow \beta$, we define a materialized view that computes a join of all relations in the decomposition, and projects the result on $\alpha\beta$. The functional dependency can be easily tested on the materialized view, by means of a constraint **unique** (α). On the negative side, there is a space and time overhead due to the materialized view, but on the positive side, the application programmer need not worry about writing code to keep redundant data consistent on updates; it is the job of the database system to maintain the materialized view, that is, keep up up to date when the database is updated. (Later in the book, in Section 14.5, we outline how a database system can perform materialized view maintenance efficiently.)

Thus, in case we are not able to get a dependency-preserving BCNF decomposition, it is generally preferable to opt for BCNF, and use techniques such as materialized views to reduce the cost of checking functional dependencies.

7.8 Fourth Normal Form

Some relation schemas, even though they are in BCNF, do not seem to be sufficiently normalized, in the sense that they still suffer from the problem of repetition of information. Consider again our banking example. Assume that, in an alternative design for the bank database schema, we have the schema

$$\textit{BC-schema} = (\textit{loan-number}, \textit{customer-name}, \textit{customer-street}, \textit{customer-city})$$

The astute reader will recognize this schema as a non-BCNF schema because of the functional dependency

$$\textit{customer-name} \rightarrow \textit{customer-street customer-city}$$

that we asserted earlier, and because *customer-name* is not a key for *BC-schema*. However, assume that our bank is attracting wealthy customers who have several addresses (say, a winter home and a summer home). Then, we no longer wish to enforce the functional dependency *customer-name* $\rightarrow$ *customer-street customer-city*. If we remove this functional dependency, we find *BC-schema* to be in BCNF with respect to our modified set of functional dependencies. Yet, even though *BC-schema* is now in BCNF, we still have the problem of repetition of information that we had earlier.

To deal with this problem, we must define a new form of constraint, called a *multivalued dependency*. As we did for functional dependencies, we shall use multivalued dependencies to define a normal form for relation schemas. This normal form, called **fourth normal form** (4NF), is more restrictive than BCNF. We shall see that every 4NF schema is also in BCNF, but there are BCNF schemas that are not in 4NF.

7.8.1 Multivalued Dependencies

Functional dependencies rule out certain tuples from being in a relation. If $A \rightarrow B$, then we cannot have two tuples with the same A value but different B values. Multivalued dependencies, on the other hand, do not rule out the existence of certain tuples. Instead, they *require* that other tuples of a certain form be present in the relation. For this reason, functional dependencies sometimes are referred to as **equality-generating dependencies**, and multivalued dependencies are referred to as **tuple-generating dependencies**.

Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if, in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that

$$\begin{aligned}
&t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha] \\
&t_3[\beta] = t_1[\beta] \\
&t_3[R - \beta] = t_2[R - \beta] \\
&t_4[\beta] = t_2[\beta] \\
&t_4[R - \beta] = t_1[R - \beta]
\end{aligned}$$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Figure 7.16 Tabular representation of $\alpha \twoheadrightarrow \beta$.

This definition is less complicated than it appears to be. Figure 7.16 gives a tabular picture of t_1, t_2, t_3, and t_4. Intuitively, the multivalued dependency $\alpha \twoheadrightarrow \beta$ says that the relationship between α and β is independent of the relationship between α and $R - \beta$. If the multivalued dependency $\alpha \twoheadrightarrow \beta$ is satisfied by all relations on schema R, then $\alpha \twoheadrightarrow \beta$ is a *trivial* multivalued dependency on schema R. Thus, $\alpha \twoheadrightarrow \beta$ is trivial if $\beta \subseteq \alpha$ or $\beta \cup \alpha = R$.

To illustrate the difference between functional and multivalued dependencies, we consider the *BC-schema* again, and the relation *bc* (*BC-schema*) of Figure 7.17. We must repeat the loan number once for each address a customer has, and we must repeat the address for each loan a customer has. This repetition is unnecessary, since the relationship between a customer and his address is independent of the relationship between that customer and a loan. If a customer (say, Smith) has a loan (say, loan number L-23), we want that loan to be associated with all Smith's addresses. Thus, the relation of Figure 7.18 is illegal. To make this relation legal, we need to add the tuples (L-23, Smith, Main, Manchester) and (L-27, Smith, North, Rye) to the *bc* relation of Figure 7.18.

Comparing the preceding example with our definition of multivalued dependency, we see that we want the multivalued dependency

$$\textit{customer-name} \twoheadrightarrow \textit{customer-street customer-city}$$

to hold. (The multivalued dependency *customer-name* $\twoheadrightarrow$ *loan-number* will do as well. We shall soon see that they are equivalent.)

As with functional dependencies, we shall use multivalued dependencies in two ways:

1. To test relations to determine whether they are legal under a given set of functional and multivalued dependencies
2. To specify constraints on the set of legal relations; we shall thus concern ourselves with *only* those relations that satisfy a given set of functional and multivalued dependencies

loan-number	*customer-name*	*customer-street*	*customer-city*
L-23	Smith	North	Rye
L-23	Smith	Main	Manchester
L-93	Curry	Lake	Horseneck

Figure 7.17 Relation *bc*: An example of redundancy in a BCNF relation.

loan-number	*customer-name*	*customer-street*	*customer-city*
L-23	Smith	North	Rye
L-27	Smith	Main	Manchester

Figure 7.18 An illegal *bc* relation.

Note that, if a relation r fails to satisfy a given multivalued dependency, we can construct a relation r' that *does* satisfy the multivalued dependency by adding tuples to r.

Let D denote a set of functional and multivalued dependencies. The **closure** D^+ of D is the set of all functional and multivalued dependencies logically implied by D. As we did for functional dependencies, we can compute D^+ from D, using the formal definitions of functional dependencies and multivalued dependencies. We can manage with such reasoning for very simple multivalued dependencies. Luckily, multivalued dependencies that occur in practice appear to be quite simple. For complex dependencies, it is better to reason about sets of dependencies by using a system of inference rules. (Section C.1.1 of the appendix outlines a system of inference rules for multivalued dependencies.)

From the definition of multivalued dependency, we can derive the following rule:

- If $\alpha \rightarrow \beta$, then $\alpha \twoheadrightarrow \beta$.

In other words, every functional dependency is also a multivalued dependency.

7.8.2 Definition of Fourth Normal Form

Consider again our *BC-schema* example in which the multivalued dependency *customer-name* $\twoheadrightarrow$ *customer-street customer-city* holds, but no nontrivial functional dependencies hold. We saw in the opening paragraphs of Section 7.8 that, although *BC-schema* is in BCNF, the design is not ideal, since we must repeat a customer's address information for each loan. We shall see that we can use the given multivalued dependency to improve the database design, by decomposing *BC-schema* into a **fourth normal form** decomposition.

A relation schema R is in **fourth normal form** (4NF) with respect to a set D of functional and multivalued dependencies if, for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds

- $\alpha \twoheadrightarrow \beta$ is a trivial multivalued dependency.
- α is a superkey for schema R.

A database design is in 4NF if each member of the set of relation schemas that constitutes the design is in 4NF.

Note that the definition of 4NF differs from the definition of BCNF in only the use of multivalued dependencies instead of functional dependencies. Every 4NF schema is in BCNF. To see this fact, we note that, if a schema R is not in BCNF, then there is

```
result := {R};
done := false;
compute D+; Given schema Ri, let Di denote the restriction of D+ to Ri
while (not done) do
    if (there is a schema Ri in result that is not in 4NF w.r.t. Di)
      then begin
              let α →→ β be a nontrivial multivalued dependency that holds
              on Ri such that α → Ri is not in Di, and α ∩ β = ∅;
              result := (result − Ri) ∪ (Ri − β) ∪ (α, β);
           end
      else done := true;
```

Figure 7.19 4NF decomposition algorithm.

a nontrivial functional dependency $\alpha \rightarrow \beta$ holding on R, where α is not a superkey. Since $\alpha \rightarrow \beta$ implies $\alpha \twoheadrightarrow \beta$, R cannot be in 4NF.

Let R be a relation schema, and let $R_1, R_2, \ldots, R_n$ be a decomposition of R. To check if each relation schema R_i in the decomposition is in 4NF, we need to find what multivalued dependencies hold on each R_i. Recall that, for a set F of functional dependencies, the restriction F_i of F to R_i is all functional dependencies in F^+ that include *only* attributes of R_i. Now consider a set D of both functional and multivalued dependencies. The **restriction** of D to R_i is the set D_i consisting of

1. All functional dependencies in D^+ that include only attributes of R_i
2. All multivalued dependencies of the form

$$\alpha \twoheadrightarrow \beta \cap R_i$$

where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+.

7.8.3 Decomposition Algorithm

The analogy between 4NF and BCNF applies to the algorithm for decomposing a schema into 4NF. Figure 7.19 shows the 4NF decomposition algorithm. It is identical to the BCNF decomposition algorithm of Figure 7.13, except that it uses multivalued, instead of functional, dependencies and uses the restriction of D^+ to R_i.

If we apply the algorithm of Figure 7.19 to *BC-schema*, we find that *customer-name* $\twoheadrightarrow$ *loan-number* is a nontrivial multivalued dependency, and *customer-name* is not a superkey for *BC-schema*. Following the algorithm, we replace *BC-schema* by two schemas:

Borrower-schema = (*customer-name*, *loan-number*)
Customer-schema = (*customer-name*, *customer-street*, *customer-city*).

This pair of schemas, which is in 4NF, eliminates the problem we encountered earlier with the redundancy of *BC-schema*.

As was the case when we were dealing solely with functional dependencies, we are interested in decompositions that are lossless-join decompositions and that preserve dependencies. The following fact about multivalued dependencies and lossless joins shows that the algorithm of Figure 7.19 generates only lossless-join decompositions:

- Let R be a relation schema, and let D be a set of functional and multivalued dependencies on R. Let R_1 and R_2 form a decomposition of R. This decomposition is a lossless-join decomposition of R if and only if at least one of the following multivalued dependencies is in D^+:

$$R_1 \cap R_2 \twoheadrightarrow R_1$$
$$R_1 \cap R_2 \twoheadrightarrow R_2$$

Recall that we stated in Section 7.5.1 that, if $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$, then R_1 and R_2 are a lossless-join decomposition of R. The preceding fact about multivalued dependencies is a more general statement about lossless joins. It says that, for *every* lossless-join decomposition of R into two schemas R_1 and R_2, one of the two dependencies $R_1 \cap R_2 \twoheadrightarrow R_1$ or $R_1 \cap R_2 \twoheadrightarrow R_2$ must hold.

The issue of dependency preservation when we decompose a relation becomes more complicated in the presence of multivalued dependencies. Section C.1.2 of the appendix pursues this topic.

7.9 More Normal Forms

The fourth normal form is by no means the "ultimate" normal form. As we saw earlier, multivalued dependencies help us understand and tackle some forms of repetition of information that cannot be understood in terms of functional dependencies. There are types of constraints called **join dependencies** that generalize multivalued dependencies, and lead to another normal form called **project-join normal form (PJNF)** (PJNF is called **fifth normal form** in some books). There is a class of even more general constraints, which leads to a normal form called **domain-key normal form**.

A practical problem with the use of these generalized constraints is that they are not only hard to reason with, but there is also no set of sound and complete inference rules for reasoning about the constraints. Hence PJNF and domain-key normal form are used quite rarely. Appendix C provides more details about these normal forms.

Conspicuous by its absence from our discussion of normal forms is **second normal form** (2NF). We have not discussed it, because it is of historical interest only. We simply define it, and let you experiment with it in Exercise 7.26.

7.10 Overall Database Design Process

So far we have looked at detailed issues about normal forms and normalization. In this section we study how normalization fits into the overall database design process.

Earlier in the chapter, starting in Section 7.4, we assumed that a relation schema R is given, and proceeded to normalize it. There are several ways in which we could have come up with the schema R:

1. R could have been generated when converting a E-R diagram to a set of tables.
2. R could have been a single relation containing *all* attributes that are of interest. The normalization process then breaks up R into smaller relations.
3. R could have been the result of some ad hoc design of relations, which we then test to verify that it satisfies a desired normal form.

In the rest of this section we examine the implications of these approaches. We also examine some practical issues in database design, including denormalization for performance and examples of bad design that are not detected by normalization.

7.10.1 E-R Model and Normalization

When we carefully define an E-R diagram, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization. However, there can be functional dependencies between attributes of an entity. For instance, suppose an *employee* entity had attributes *department-number* and *department-address*, and there is a functional dependency *department-number* $\rightarrow$ *department-address*. We would then need to normalize the relation generated from *employee*.

Most examples of such dependencies arise out of poor E-R diagram design. In the above example, if we did the E-R diagram correctly, we would have created a *department* entity with attribute *department-address* and a relationship between *employee* and *department*. Similarly, a relationship involving more than two entities may not be in a desirable normal form. Since most relationships are binary, such cases are relatively rare. (In fact, some E-R diagram variants actually make it difficult or impossible to specify nonbinary relations.)

Functional dependencies can help us detect poor E-R design. If the generated relations are not in desired normal form, the problem can be fixed in the E-R diagram. That is, normalization can be done formally as part of data modeling. Alternatively, normalization can be left to the designer's intuition during E-R modeling, and can be done formally on the relations generated from the E-R model.

7.10.2 The Universal Relation Approach

The second approach to database design is to start with a single relation schema containing all attributes of interest, and decompose it. One of our goals in choosing a decomposition was that it be a lossless-join decomposition. To consider losslessness, we assumed that it is valid to talk about the join of all the relations of the decomposed database.

Consider the database of Figure 7.20, showing a decomposition of the *loan-info* relation. The figure depicts a situation in which we have not yet determined the amount of loan L-58, but wish to record the remainder of the data on the loan. If we compute the natural join of these relations, we discover that all tuples referring to loan L-58 disappear. In other words, there is no *loan-info* relation corresponding to the relations of Figure 7.20. Tuples that disappear when we compute the join are *dangling tuples* (see Section 6.2.1). Formally, let $r_1(R_1),\ r_2(R_2), \ldots, r_n(R_n)$ be a set of relations. A

branch-name	*loan-number*
Round Hill	L-58

loan-number	*amount*

loan-number	*customer-name*
L-58	Johnson

Figure 7.20 Decomposition of *loan-info*.

tuple t of relation r_i is a dangling tuple if t is not in the relation

$$\Pi_{R_i} \ (r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n)$$

Dangling tuples may occur in practical database applications. They represent incomplete information, as they do in our example, where we wish to store data about a loan that is still in the process of being negotiated. The relation $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$ is called a **universal relation**, since it involves all the attributes in the universe defined by $R_1 \cup R_2 \cup \cdots \cup R_n$.

The only way that we can write a universal relation for the example of Figure 7.20 is to include *null values* in the universal relation. We saw in Chapter 3 that null values present several difficulties. Because of them, it may be better to view the relations of the decomposed design as representing the database, rather than as the universal relation whose schema we decomposed during the normalization process. (The bibliographical notes discuss research on null values and universal relations.)

Note that we cannot enter all incomplete information into the database of Figure 7.20 without resorting to null values. For example, we cannot enter a loan number unless we know at least one of the following:

- The customer name
- The branch name
- The amount of the loan

Thus, a particular decomposition defines a restricted form of incomplete information that is acceptable in our database.

The normal forms that we have defined generate good database designs from the point of view of representation of incomplete information. Returning again to the example of Figure 7.20, we would not want to allow storage of the following fact: "There is a loan (whose number is unknown) to Jones in the amount of $100." This is because

$$\textit{loan-number} \rightarrow \textit{customer-name amount}$$

and therefore the only way that we can relate *customer-name* and *amount* is through *loan-number*. If we do not know the loan number, we cannot distinguish this loan from other loans with unknown numbers.

In other words, we do not want to store data for which the key attributes are unknown. Observe that the normal forms that we have defined do not allow us to store that type of information unless we use null values. Thus, our normal forms allow representation of acceptable incomplete information via dangling tuples, while prohibiting the storage of undesirable incomplete information.

Another consequence of the universal relation approach to database design is that attribute names must be unique in the universal relation. We cannot use *name* to refer to both *customer-name* and to *branch-name*. It is generally preferable to use unique names, as we have done. Nevertheless, if we defined our relation schemas directly, rather than in terms of a universal relation, we could obtain relations on schemas such as the following for our banking example:

branch-loan (*name, number*)
loan-customer (*number, name*)
amt (*number, amount*)

Observe that, with the preceding relations, expressions such as *branch-loan* $\bowtie$ *loan-customer* are meaningless. Indeed, the expression *branch-loan* $\bowtie$ *loan-customer* finds loans made by branches to customers who have the same name as the name of the branch.

In a language such as SQL, however, a query involving *branch-loan* and *loan-customer* must remove ambiguity in references to *name* by prefixing the relation name. In such environments, the multiple roles for *name* (as branch name and as customer name) are less troublesome and may be simpler to use.

We believe that using the **unique-role assumption**—that each attribute name has a unique meaning in the database—is generally preferable to reusing of the same name in multiple roles. When the unique-role assumption is not made, the database designer must be especially careful when constructing a normalized relational-database design.

7.10.3 Denormalization for Performance

Occasionally database designers choose a schema that has redundant information; that is, it is not normalized. They use the redundancy to improve performance for specific applications. The penalty paid for not using a normalized schema is the extra work (in terms of coding time and execution time) to keep redundant data consistent.

For instance, suppose that the name of an account holder has to be displayed along with the account number and balance, every time the account is accessed. In our normalized schema, this requires a join of *account* with *depositor*.

One alternative to computing the join on the fly is to store a relation containing all the attributes of *account* and *depositor*. This makes displaying the account information faster. However, the balance information for an account is repeated for every person who owns the account, and all copies must be updated by the application, whenever the account balance is updated. The process of taking a normalized schema and making it non-normalized is called **denormalization**, and designers use it to tune performance of systems to support time-critical operations.